

M6: Production Release

Overview

The final milestone is the *production release*, which should include the final, refined version of your project which will be widely distributed. As the name implies, the production release must be a complete product with all final features integrated, functional, polished, and thoroughly bug tested. Implementation of all aspects of every feature must be completed, along with multiple rounds of user testing. The production release is the final version of the application that is delivered to clients, and as such it should be usable by a typical user. The production release build should represent approximately 150 hours of work per team member, or about 450-750 person-hours for the entire team (not including design documentation).

Specification

The deliverable should meet the following requirements.

User Experience

The user feedback system, including UI and sensory response elements, should complete and polished in the production release. All elements of user control should be completed.

Interface

All interface elements should be visible, and usable, and polished. Options should be intuitive and consistent. Additionally, any persistent state must be easily accessible (e.g., illumination of LEDs and/or state API calls). For user interfaces (UIs), selected button(s), menu option(s), or other dialog element(s) should be highlighted.

Navigation

All user mechanics should be functional and polished. Any controls should be thoroughly tested. There should be no bugs. Controls should not require significant time for acclimation; they should be predictable and easy to use. Features should be easily discoverable and well-explained. They should not overwhelm the new users.

Perception

The artifact must be intuitive to users. Teams should have solicited user testing from non-team members to ensure general usability. Users should report an enjoyable experience. Any control interactions must be indicated by the sensory experience (visually and/or audibly) in an intuitive manner. The artifact should help users attain stated goals with minimal frustration.

Responsiveness

The artifact must be responsive to user input. Changes in application state should be indicated on or near the relevant interface elements. For any operation that requires significant I/O operations, the call should yield the CPU while awaiting response – i.e., there should be no busy wait loops. For APIs, if any call may have a delayed return, there should be a non-blocking option within the API call. Completion or failure of tasks must be clearly indicated to the user.

Build Quality

Projects receiving a perfect build score will have no detectable bugs. All content, including art and code systems, should be present and polished.

Robustness

There should be no crashes or glitches in the production release of the project. The system should gracefully handle failures from user input, corrupted data, and/or inconsistent information.

Consistency

Except where explicitly by design, the application should act predictably – for the same input / use case, it should yield the same result. When / if behavior is unpredictable, it must be for a clear and compelling reason.

Aesthetic Rigor

No cosmetic issues should be present. Under no circumstances should aesthetic issues cause the application to be unstable, frustrating, or unusable. All assets (e.g., images and sounds) should be present and functional within the context of the project. Assets and their integration within the application should be polished.

Features

All features should be complete, with edge cases tested and accounted for, and with failure modes anticipated and handled.

External Interface

For each possible use-case, control scheme(s) must be implemented, usable, and smooth within the artifact. State changes must be reflected by the artifact. This must connect to the *persistent state*.

Persistent State

For each possible use case, the release should make proper use of a data store to update the state of the system. This must connect to the *external interface* and the *persistent state*.

Internal Systems

For each possible use case, all data processing steps must be fully implemented. This must connect to the *persistent state*.

Submissions

The submission must include the following components:

- A brief report (approximately 1-2 pages) in PDF format that outlines the work done as it relates to the specification and rubric criteria.
- A link to the project repository (in the report).
- A presentation and defense of the work completed as it relates to the project specification and milestone rubric criteria. Your group must present and defend in-person to your Stakeholder in the week following the assigned due date (**you are responsible for setting up that meeting**).

In addition to all source code and hardware designs, the project repository should include a README outlining completed work, a description of the project's architecture, and all known bugs. **Failure to document bugs is grounds for grade reduction.**

It is also *critical* that all teams have time-stamped third-party evidence (e.g., remote source code repository pushes, online documents, etc.; action logging is **not** acceptable as it doesn't constitute proof of work) of all effort invested in the project. Teams may include additional documentation, as necessary, to demonstrate the work completed. **Failure to establish sufficient evidence to prove effort investment will result in a proportional grade deduction.**